

AIQT Milestone 10 Build Specification v0.3

Final Review Classification Hardening and Integration Skills Bootstrap

Status: Build-ready draft. Scope is intentionally bounded: harden the M9 final-review/manager classification model and add read-only integration skill planning for Supabase, Clerk, and shadcn/ui.

1. Source Alignment

- Depends on M9 merged to main: workflow manager, review acknowledgment, review modes, next preview/cancel, and final-review export.
- Preserves AIQT core contract: local state engine, structured canonical state, append-only runlog, generated exports only.
- M10 must not introduce AI calls, remote connector calls, package publishing, automatic skill installation, or graph/checkpoint mutation commands.
- M10 addresses M9 dogfood follow-up findings: final-review misclassification, narrow release blockers, narrow external verification gaps, weak recommendation loop, and integration friction for Supabase/Clerk/shadcn/ui.

2. Objective

M10 has two bounded goals. First, harden final-review/manage classification so terminal project reports are useful for release decisions. Second, add read-only integration skills planning so AIQT can recommend relevant public agent skills without installing or executing them.

3. Problems to Solve

- final-review.md currently lists some non-user-action issues under User-Action-Required.
- External Verification Gaps only includes review findings, missing many checkpoint issues that clearly require live service verification.
- Release Blockers are too narrowly derived from blocking review findings.
- Post-MVP Backlog Candidates can be empty even when low/medium unresolved issues exist.
- Recommended next action can point back to review even when rerunning review will repeat the same blocker.
- Supabase, Clerk, and shadcn/ui integrations would be faster and safer if AIQT could generate a deterministic skills bootstrap plan.

4. In Scope

- Improve final-review.md classification sections generated by aiqt export all.
- Update aiqt manage to use the same classification service as final-review.md.
- Extend centralized nextRecommendedCommand logic with terminal/release cases; keep the field single-valued.
- Add aiqt skills plan and aiqt skills plan --json as read-only commands.
- Detect Supabase, Clerk, and shadcn/ui from project context and repository files.
- Add integration skill hints to future agent packets when relevant and when skills are detected/recommended.
- Add tests for classification consistency, recommendations, skills detection, and export output.

5. Out of Scope

- No aiqt skills install command in M10.
- No external installer execution: do not run npx skills add or pnpm dlx skills add.
- No checkpoint amend, dependency update, issue promote, repair plan, or graph amendment.

- No rich checkpoint issue schema migration. M10 uses deterministic heuristics and existing fields.
- No standalone aigt export final-review target. final-review.md remains generated by aigt export all only.
- No requirement lifecycle management and no package publishing.

6. Repository Implementation Scope

- Likely command files: src/commands/skills*.ts, existing export/manage/review command modules, CLI registration.
- Likely core files: classification service for final review/manage, next-action recommendation service, packet template helpers.
- Likely schema files: skills plan output schemas if existing conventions require them. No AIQT_SCHEMA_VERSION bump expected.
- Likely export files: export-service and export-templates for final-review.md classification sections.
- Likely tests: skills command tests, final-review export tests, manage/export consistency tests, next-action tests, packet hint tests.

7. Runtime and Package Impact

- No new runtime package dependency expected.
- No network access during AIQT command execution.
- No filesystem mutation for aigt skills plan. It is read-only and must not append runlog events.
- aigt export all continues to write generated files under .aiqt/exports/.
- The implementation may read package.json, lockfiles, project context, and existing repository paths for deterministic skills detection.

8. Final Review Classification Contract

8.1 Export target decision

M10 does not add a standalone final-review export target. final-review.md remains generated by aigt export all. Adding aigt export final-review would require extending the export target enum, CLI help, target validation, and tests; this is explicitly out of scope.

8.2 Classification precedence

CheckpointIssue.agentCanFix is authoritative where present. Heuristics only fill classification gaps.

- If agentCanFix === false, the issue must not be classified as agent-fixable, regardless of keywords such as RLS, schema, query, code, or form.
- If agentCanFix === true, the issue may be classified as agent-fixable unless a stronger user-action/external-verification signal exists.
- Explicit user-action wording, live service setup, credentials, dashboard configuration, deployment, smoke tests, branch protection, or legal/privacy review should classify as user-action-required or release-blocking as applicable.
- If no explicit field resolves classification, deterministic text heuristics may classify the issue into one or more final-review buckets.

8.3 Required classification buckets

- User Action Required: operator/user-controlled setup or action, such as Supabase/Clerk/Resend/Sentry/Vercel setup, Docker availability, branch protection, legal/privacy/GDPR review, live smoke tests.
- External Verification Gaps: live runtime verification, credentials, provider delivery, Storage/Clerk/Supabase/Resend/Sentry/Vercel verification, deployment and smoke-test gaps.
- Agent-Fixable Unresolved Issues: issues explicitly agentCanFix true, or heuristic technical issues not requiring external/user setup.

- Release Blockers: blocking review findings, required live setup/verification, branch protection/CI enforcement, legal/privacy review, and medium/high unresolved issues classified as release-blocking.
- Post-MVP Backlog Candidates: low/medium issues suitable for later optimization or non-MVP hardening, unless release-blocking by severity/category.

8.4 Non-user-action examples

The following must not appear in User Action Required unless explicitly marked as user action: scope notes, implementation trade-offs, fixed-dictionary i18n limitations, audit-scope notes, and internal optimization notes.

9. Manage and Export Consistency

aiqt manage and final-review.md must use the same classification service. Human formatting may differ, but classification values must match exactly.

- developmentComplete must match.
- productionReady must match.
- releaseBlockerCount must match.
- userActionRequiredCount must match.
- externalVerificationGapCount must match.
- agentFixableIssueCount must match.

This is a testable contract, not a soft guideline.

10. Centralized nextRecommendedCommand Contract

M10 must extend the existing centralized nextRecommendedCommand mechanism. Do not scatter recommendation logic across exports, manage, review, and status.

10.1 Single-value rule

state.nextRecommendedCommand remains exactly one command string. Multi-step guidance belongs in human-readable reason text, not in the canonical field.

```
Case: development complete, production not ready, release blockers exist nextRecommendedCommand = "aiqt manage" reason = "Resolve release blockers, then rerun aiqt review --mode release." Case: development complete, production readiness unknown due to missing user setup/live verification nextRecommendedCommand = "aiqt manage" reason = "Complete user-action-required setup and live verification." Case: all work units done, no unacknowledged development blockers, final exports not yet generated nextRecommendedCommand = "aiqt export all" reason = "Development-complete export is available." Case: release review failed with unacknowledged development blockers nextRecommendedCommand = "aiqt review acknowledge" reason = "Acknowledge accepted historical findings or fix the blocker."
```

11. Skills Plan Command

11.1 Command

```
aiqt skills plan aiqt skills plan --json
```

The command is read-only. It must not mutate state, write files, install skills, or append runlog events.

11.2 Supported integrations

Internal JSON id	Display name	Detection hints	Recommended install command
supabase	Supabase	package/dependency names, supabase/ directory, migrations, project context mentions	npx skills add supabase/agent-skills
clerk	Clerk	package/dependency names, middleware/auth/webhook files, project context mentions	npx skills add clerk/skills

shadcn-ui	shadcn/ui	components.json, shadcn/ui dependencies, app/components patterns, project context mentions	pnpm dlx skills add shadcn/ui
-----------	-----------	--	-------------------------------

Integration display names may contain slashes, but JSON ids use stable lowercase slugs. Therefore display name shadcn/ui maps to JSON id shadcn-ui.

11.3 Confidence model

- high: direct repository evidence such as dependency plus configuration/directory evidence.
- medium: strong project-context evidence or dependency evidence only.
- low: weak textual mention only; should recommend review before use.

11.4 Deterministic JSON output

```
{ "detectedIntegrations": [ { "id": "supabase", "displayName": "Supabase", "confidence": "high",
"evidence": ["package.json dependency @supabase/supabase-js", "supabase/migrations directory"],
"recommendedSkill": { "id": "supabase-agent-skills", "installCommand": "npx skills add
supabase/agent-skills", "installAutomatically": false } }, { "id": "clerk", "displayName": "Clerk",
"confidence": "high", "evidence": ["package.json dependency @clerk/nextjs", "middleware.ts references
Clerk"], "recommendedSkill": { "id": "clerk-skills", "installCommand": "npx skills add clerk/skills",
"installAutomatically": false } }, { "id": "shadcn-ui", "displayName": "shadcn/ui", "confidence":
"high", "evidence": ["components.json", "src/components/ui directory"], "recommendedSkill": { "id":
"shadcn-ui", "installCommand": "pnpm dlx skills add shadcn/ui", "installAutomatically": false } } ],
"notDetectedIntegrations": [], "safetyNotes": [ "AIQT does not install external skills
automatically.", "Review public skill contents before relying on them.", "AIQT packet scope and
project constraints override generic skill guidance." ] }
```

Ordering must be deterministic: supabase, clerk, shadcn-ui, then future integrations alphabetically by id. Evidence arrays must be deterministic and stable.

12. Agent Packet Skill Hints

When a future packet touches an integration with a detected/recommended skill, the packet may include a concise Relevant Skills section. Packet scope remains authoritative. Skills are advisory only.

```
Relevant Skills: - Supabase agent skills may help with migrations, RLS, Storage, and local validation.
Install command: npx skills add supabase/agent-skills - Clerk skills may help with auth, middleware,
sessions, JWT claims, and webhooks. Install command: npx skills add clerk/skills
Priority: 1. AIQT packet scope and out-of-scope 2. Project-specific constraints 3. Current repository code 4.
Installed/recommended skills 5. Generic model knowledge
```

13. Exit Code Contract

Command	Exit 0	Exit 1	Exit 2	Exit 3
aiqt skills plan	Plan generated	n/a	n/a	Invalid project/state or unreadable files needed for detection
aiqt manage	Report generated	n/a	n/a	Invalid project/state
aiqt export all	Exports generated, including final-review.md	n/a	n/a	Invalid target/state/export failure
aiqt review --mode release	No release blockers	Release blockers exist	n/a	Invalid mode/state
aiqt review --mode development	No unacknowledged development blockers	Unacknowledged development blockers exist	n/a	Invalid mode/state

14. Test Matrix

14.1 Unit tests

- Classification service respects agentCanFix precedence.
- External verification gap detection catches Supabase/Clerk/Storage/Resend/Sentry/Vercel/live/smoke-test language.

- User-action classifier excludes scope notes and technical limitations that are not user actions.
- Post-MVP/backlog classifier captures low/medium optimization notes.
- Skills detection confidence is deterministic.
- shadcn/ui display name maps to shadcn-ui JSON id.

14.2 Integration tests

- aiqt export all final-review.md includes cleaned user-action, external verification, release blockers, and backlog sections.
- aiqt manage and final-review.md produce identical classification counts and booleans.
- nextRecommendedCommand remains a single command string for terminal/release states.
- aiqt skills plan --json output matches the JSON contract and deterministic order.

14.3 Regression tests

- Completed dogfood-style state: developmentComplete yes, productionReady no, acknowledged WU003 finding visible, user-action-required list excludes non-user scope notes.
- Final-review Release Blockers includes live smoke-test/deployment/branch protection/legal setup when present.
- External Verification Gaps includes checkpoint issues, not only review findings.

14.4 CLI contract tests

- aiqt skills plan exits 0 in text and JSON mode.
- aiqt skills plan does not mutate state or runlog.
- aiqt export all remains the only command generating final-review.md.
- No aiqt export final-review command is introduced.

15. Definition of Done

- final-review.md no longer misclassifies non-user-action issues as user-action-required.
- External Verification Gaps includes relevant checkpoint issues and review findings.
- Release Blockers include release-relevant setup, verification, legal/compliance, and critical unresolved issues, not only blocking review findings.
- Post-MVP Backlog Candidates are populated when low/medium non-blocking issues exist.
- Recommended next command remains a deterministic single command string.
- aiqt manage and final-review.md classification values match exactly.
- aiqt skills plan and aiqt skills plan --json are implemented and read-only.
- Supabase, Clerk, and shadcn/ui are detected with deterministic confidence and evidence.
- Skills plan recommends the exact install commands without executing them.
- Agent packets include concise integration skill hints when relevant.
- No state schema version bump is required unless implementation introduces incompatible changes.
- All existing tests pass and new tests cover classification, recommendations, skills planning, exports, and packet hints.

16. Claude Code Handoff Summary

Implement M10 on top of merged M9. Keep the milestone bounded: classification hardening plus read-only skills plan. Do not add skill installation or repair commands. Use existing centralized classification/recommendation services where possible and avoid scattering terminal-state logic.